Lab 9: Estimating and Stress-Testing an RD

SOCIOL 723, Social Statistics II, Thursday, October 22

Wenhao Jiang

Table of contents

1 Goals

1. Draw the picture first, because in an RD the picture *is* the result.
2. Estimate with local linear regression and robust bias-corrected intervals.
3. Choose a bandwidth, and see how much the answer depends on it.
4. Run every validity check an RD paper should report.
5. See in simulation what manipulation of the running variable does.

```
library(dplyr)
library(ggplot2)
library(rdrobust)
library(rddensity)

data("rdrobust_RDsenate", package = "rdrobust")
sen <- rdrobust_RDsenate |> filter(!is.na(vote))
c(n = nrow(sen), min_margin = min(sen$margin), max_margin = max(sen$margin))
#>          n min_margin max_margin
#>       1297       -100        100
```

2 The design

Question. Does winning a Senate seat make a party more likely to win that seat again six years later — the *incumbency advantage*?

Running variable. `margin`: the Democratic vote-share margin of victory in the previous election. **Cutoff.** Zero. **Treatment.** Democratic incumbency. **Outcome.** `vote`: the Democratic vote share in the following election.

The identification claim is that elections decided by a fraction of a percentage point are essentially coin flips, so parties that barely won and parties that barely lost are comparable in everything except incumbency.

3 The picture comes first

```
rdplot(y = sen$vote, x = sen$margin, c = 0,
       x.label = "Democratic margin of victory, previous election",
       y.label = "Democratic vote share, next election",
       title = "")
```

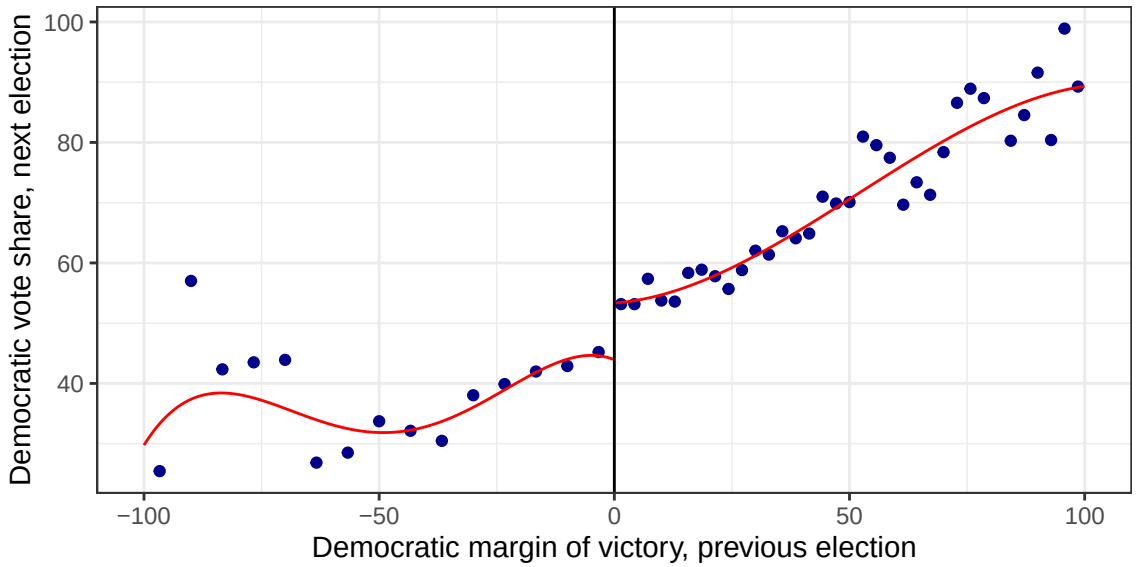

Figure 1: Binned means of the following election's Democratic vote share against the previous margin of victory, with fourth-order polynomials fitted separately on each side.

> **!** Important
>
> Look at the jump at zero before you compute anything. If you cannot see it here, no amount of bandwidth tuning should convince you it is there. Conversely, a jump this visible does not need a high-order polynomial to find it.

```
rdplot(y = sen$vote[abs(sen$margin) < 15], x = sen$margin[abs(sen$margin) < 15],
       c = 0, p = 1, nbins = c(20, 20),
       x.label = "margin", y.label = "next vote share", title = "")
```

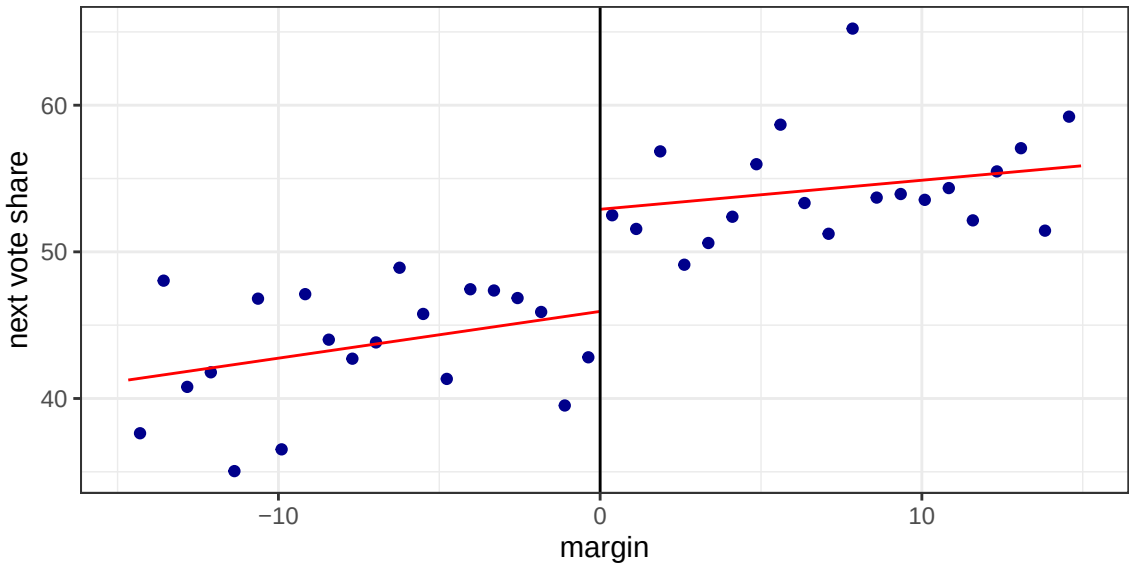

Figure 2: The same data with local linear fits inside a 15-point window. This is closer to what the estimator actually uses.

4 Estimation

```
rd <- rdrobust(y = sen$vote, x = sen$margin, c = 0)
summary(rd)
#> Sharp RD estimates using local polynomial regression.
#>
#> Number of Obs.                 1297
#> BW type                       mserd
#> Kernel                   Triangular
#> VCE method                       NN
#>
#> Number of Obs.                 595         702
#> Eff. Number of Obs.            360         323
#> Order est. (p)                   1           1
#> Order bias  (q)                  2           2
#> BW est. (h)                 17.754      17.754
#> BW bias (b)                 28.028      28.028
#> rho (h/b)                    0.633       0.633
#> Unique Obs.                    595         665
#>
#> ===============================================================================
#>                     Point    Robust Inference
#>                Estimate         z     P>|z|      [ 95% C.I. ]
#> -------------------------------------------------------------------------------
```

```
#>      RD Effect     7.414     4.311     0.000     [4.094 , 10.919]
#> ================================================================
```

Three numbers to read from that output.

- **Conventional**: the local linear estimate at the MSE-optimal bandwidth, with a naive standard error.
- **Bias-Corrected**: the point estimate after subtracting an estimate of the leading bias term.
- **Robust**: the confidence interval that accounts for the *uncertainty in that bias correction.* **This is the one to report** (Calonico, Cattaneo, and Titiunik 2014).

```
## how many observations are actually doing the work?
c(bandwidth_left = rd$bws[1, 1], bandwidth_right = rd$bws[1, 2],
  n_left = rd$N_h[1], n_right = rd$N_h[2], n_total = sum(rd$N))
#>  bandwidth_left bandwidth_right          n_left         n_right         n_total
#>           17.75           17.75          360.00          323.00         1297.00
```

Note the gap between the nominal sample size and the effective one. An RD with 1,300 observations may be estimated from 400.

4.1 Doing it by hand

`rdrobust` is a black box until you have written the estimator once.

```
h <- rd$bws[1, 1]                          # use the same bandwidth

sub <- sen |> filter(abs(margin) <= h) |>
  mutate(D = as.integer(margin >= 0),
         kw = 1 - abs(margin) / h)         # triangular kernel

## fully interacted local linear regression: separate slopes each side
fit_manual <- lm(vote ~ D * margin, data = sub, weights = kw)

c(manual = coef(fit_manual)["D"], rdrobust_conventional = rd$coef[1])
#>              manual.D rdrobust_conventional
#>                 7.414                 7.414
```

The coefficient on the treatment indicator in a kernel-weighted, fully interacted local linear regression *is* the RD estimate. Everything `rdrobust` adds is bandwidth selection and bias correction.

4.2 Bandwidth sensitivity

```
bws <- seq(3, 40, by = 1)
sens <- lapply(bws, function(b) {
```

```
  r <- rdrobust(y = sen$vote, x = sen$margin, c = 0, h = b)
  tibble::tibble(h = b, est = r$coef[3],          # bias-corrected point estimate
                 lo = r$ci[3, 1], hi = r$ci[3, 2]) # matching robust interval
}) |> bind_rows()

ggplot(sens, aes(h, est)) +
  geom_ribbon(aes(ymin = lo, ymax = hi), fill = "navy", alpha = .15) +
  geom_line(colour = "navy", linewidth = .8) +
  geom_hline(yintercept = 0, linetype = 2) +
  geom_vline(xintercept = rd$bws[1, 1], colour = "firebrick", linetype = 3) +
  labs(x = "bandwidth", y = "RD estimate") +
  theme_minimal(base_size = 10)
```

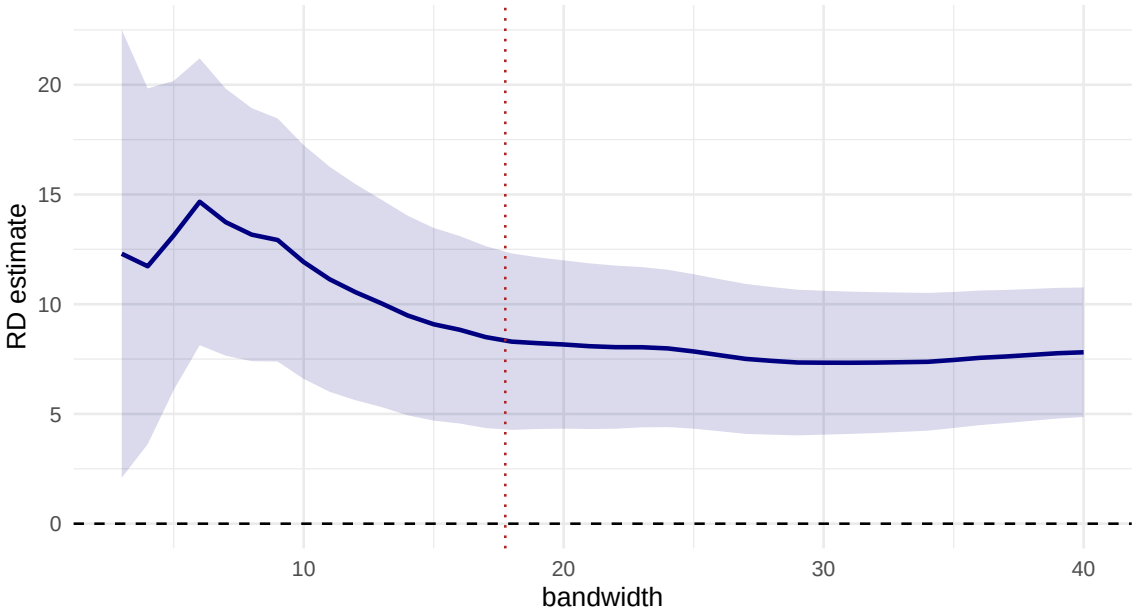

Figure 3: The RD estimate across bandwidths from 3 to 40 points, with robust 95% intervals. A credible result is flat over a wide range.

The dotted red line marks the MSE-optimal bandwidth. The estimate is stable across a wide range — this is what a robust RD looks like.

4.3 Why not a global polynomial?

```
poly_est <- sapply(1:8, function(p) {
  m <- lm(vote ~ poly(margin, p, raw = TRUE) * I(margin >= 0), data = sen)
  coef(m)["I(margin >= 0)TRUE"]
})
round(setNames(poly_est, paste0("degree_", 1:8)), 3)
```

```
#> degree_1 degree_2 degree_3 degree_4 degree_5 degree_6 degree_7 degree_8
#>    6.044    4.935    7.319    9.407    7.958    6.627    8.930    9.099
```

The answer wanders with the polynomial degree, and every one of these estimates uses observations 80 points from the cutoff to fit the value *at* the cutoff. Gelman and Imbens (2019) explain why the implied weights are indefensible.

5 Validity checks

5.1 Is the density of the running variable continuous?

If parties could precisely control their vote margin, we would see bunching just above zero.

```
dens <- rddensity(X = sen$margin, c = 0)
summary(dens)
#>
#> Manipulation testing using local polynomial density estimation.
#>
#> Number of obs =       1297
#> Model =               unrestricted
#> Kernel =              triangular
#> BW method =           estimated
#> VCE method =          jackknife
#>
#> c = 0                 Left of c           Right of c
#> Number of obs         595                 702
#> Eff. Number of obs    375                 376
#> Order est. (p)        2                   2
#> Order bias (q)        3                   3
#> BW est. (h)           18.439              22.774
#>
#> Method                T                   P > |T|
#> Robust                -0.76               0.4473
#>
#> P-values of binomial tests (H0: p=0.5).
#>
#> Window Length / 2          <c     >=c    P>|T|
#> 1.080                       20      28    0.3123
#> 2.160                       49      50    1.0000
#> 3.240                       85      73    0.3816
#> 4.320                      113     100    0.4110
#> 5.400                      136     123    0.4559
#> 6.480                      154     140    0.4484
#> 7.560                      188     159    0.1327
```

```
#> 8.640                    220     178    0.0397
#> 9.720                    240     201    0.0702
#> 10.800                   257     224    0.1445
```

```
rdplotdensity(dens, sen$margin, plotN = 100)
#> $Estl
#> Call: lpdensity
#>
#> Sample size                                    595
#> Polynomial order for point estimation    (p=)  2
#> Order of derivative estimated            (v=)  1
#> Polynomial order for confidence interval (q=)  3
#> Kernel function                                triangular
#> Scaling factor                                 0.459104938271605
#> Bandwidth method                               user provided
#>
#> Use summary(...) to show estimates.
#>
#> $Estr
#> Call: lpdensity
#>
#> Sample size                                    702
#> Polynomial order for point estimation    (p=)  2
#> Order of derivative estimated            (v=)  1
#> Polynomial order for confidence interval (q=)  3
#> Kernel function                                triangular
#> Scaling factor                                 0.541666666666667
#> Bandwidth method                               user provided
#>
#> Use summary(...) to show estimates.
#>
#> $Estplot
```

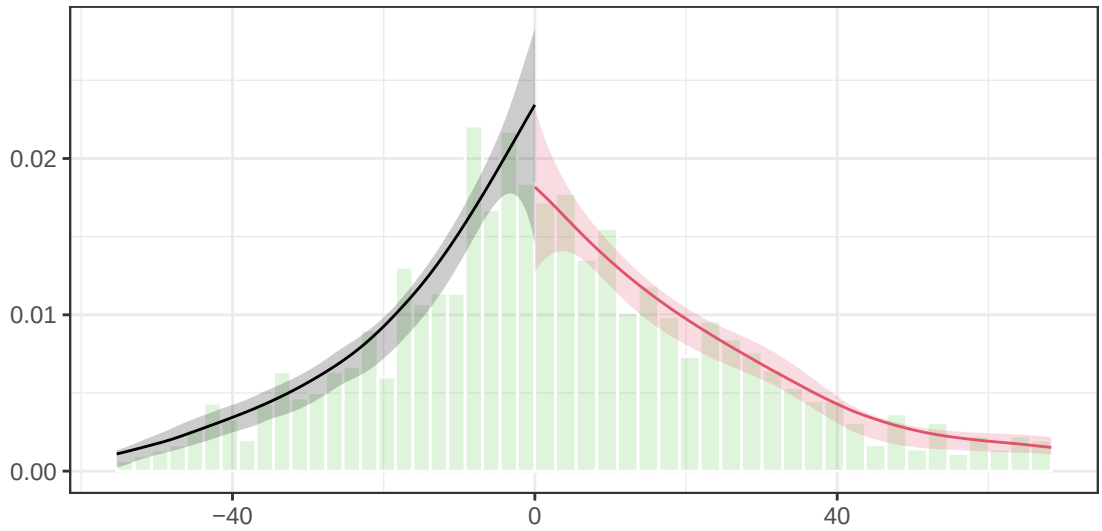

Figure 4: Estimated density of the running variable on each side of the cutoff with confidence bands. Overlapping bands at zero are what we want to see.

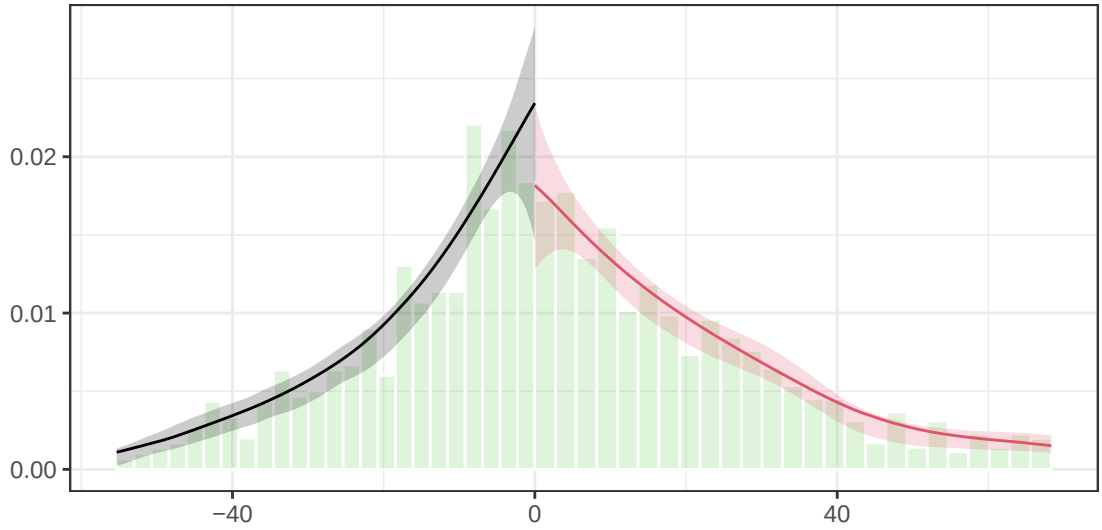

Figure 5: Estimated density of the running variable on each side of the cutoff with confidence bands. Overlapping bands at zero are what we want to see.

Non-rejection is consistent with the design, but it is not a validation of it: the test has limited power, and it detects only manipulation that shows up as bunching in the density. In settings where actors *can* manipulate the running variable (income thresholds, test retakes) treat it as one weak signal among several.

5.2 Placebo cutoffs

The estimator should find nothing at thresholds where nothing happens.

```r
placebos <- c(-20, -15, -10, -5, 5, 10, 15, 20)
plc <- sapply(placebos, function(cc) {
  d <- if (cc < 0) sen |> filter(margin < 0) else sen |> filter(margin >= 0)
  r <- rdrobust(y = d$vote, x = d$margin, c = cc)
  c(estimate = r$coef[1], p_robust = r$pv[3])
})
round(t(plc), 3) |> `rownames<-`(paste("cutoff", placebos))
#>            estimate p_robust
#> cutoff -20    6.620    0.164
#> cutoff -15   -6.893    0.233
#> cutoff -10  -23.216    0.005
#> cutoff -5    -0.177    0.722
#> cutoff 5      3.321    0.673
#> cutoff 10     2.915    0.108
#> cutoff 15    -1.218    0.648
#> cutoff 20    -0.250    0.894
```

Note the restriction to one side of the true cutoff: otherwise a placebo test would pick up the genuine discontinuity at zero.

5.3 Donut hole

Drop the observations nearest the cutoff, where manipulation would be concentrated.

```r
donut <- sapply(c(0, 0.25, 0.5, 1), function(dd) {
  d <- sen |> filter(abs(margin) > dd)
  r <- rdrobust(y = d$vote, x = d$margin, c = 0)
  c(estimate = r$coef[1], ci_lo = r$ci[3, 1], ci_hi = r$ci[3, 2])
})
round(t(donut), 3) |> `rownames<-`(paste("hole =", c(0, 0.25, 0.5, 1)))
#>             estimate ci_lo ci_hi
#> hole = 0       7.414 4.094 10.92
#> hole = 0.25    6.936 3.315 10.45
#> hole = 0.5     7.092 3.196 10.83
#> hole = 1       6.581 2.187 10.60
```

6 What manipulation actually does

Simulate a world where the treatment has **no effect at all**, but where units just below the cutoff can push themselves above it.

```r
set.seed(723)
n <- 4000
u <- rnorm(n)                                # ability: raises both X and Y
```

```r
x_true <- rnorm(n) + 0.8 * u

## units with high u who are just below the cutoff push themselves over
push <- (x_true > -0.3 & x_true < 0) & (u > 0.5)
x <- ifelse(push, abs(x_true) + 0.01, x_true)

d <- as.integer(x >= 0)
y <- 0 * d + u + rnorm(n)                  # TRUE EFFECT IS ZERO

r_bad <- rdrobust(y = y, x = x, c = 0)
c(estimate = r_bad$coef[1], p_robust = r_bad$pv[3], truth = 0)
#>  estimate  p_robust     truth
#> 0.4515959 0.0007735 0.0000000
```

```r
dt <- rddensity(X = x, c = 0)
c(t_statistic = dt$test$t_jk, p_value = dt$test$p_jk)
#> t_statistic     p_value
#>     2.19083     0.02846
```

The RD finds a large "effect" that does not exist — and **the density test catches it**. That is why the test belongs in every RD paper — while remembering that manipulation which does *not* produce bunching would slip past it.

```r
tibble::tibble(x) |> filter(abs(x) < 1.5) |>
  ggplot(aes(x)) +
  geom_histogram(bins = 60, fill = "navy", alpha = .7) +
  geom_vline(xintercept = 0, colour = "firebrick", linewidth = .8) +
  labs(x = "running variable", y = "count") +
  theme_minimal(base_size = 10)
```

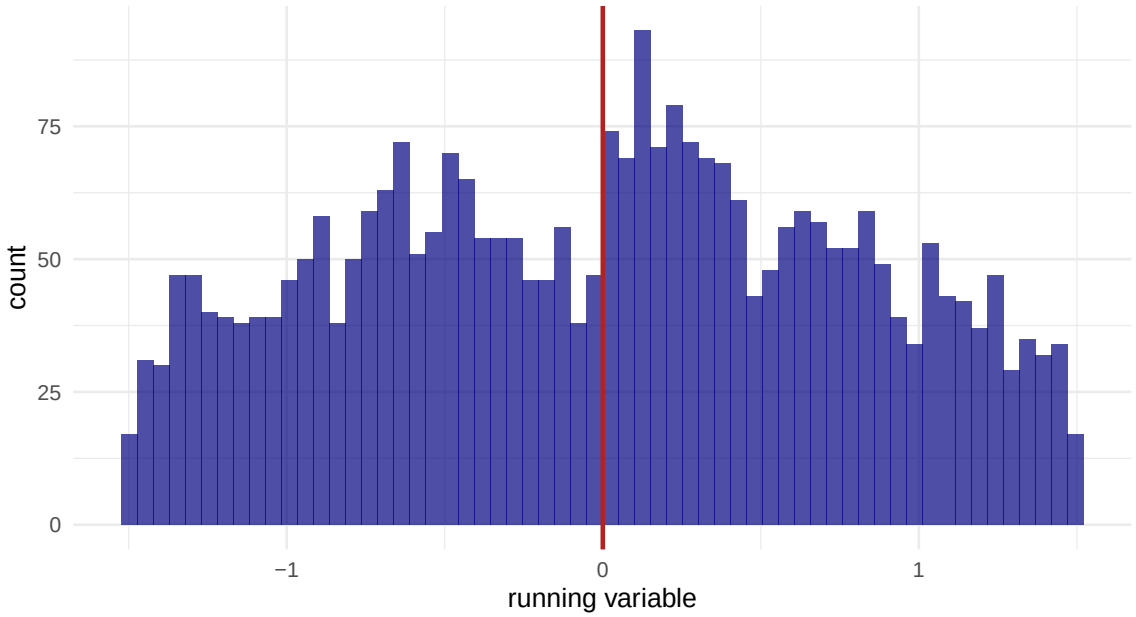

Figure 6: Histogram of the running variable under manipulation. The excess mass just above zero is the signature.

7 Fuzzy RD

When crossing the cutoff changes the *probability* of treatment, `rdrobust` takes a `fuzzy` argument and returns the ratio estimate.

```
set.seed(1)
n <- 3000
xx <- runif(n, -1, 1)
takeup <- rbinom(n, 1, ifelse(xx >= 0, 0.75, 0.20))   # jump of 0.55
yy <- 1.5 * takeup + 0.5 * xx + rnorm(n)               # TRUE effect = 1.5

## sharp (intention-to-treat): attenuated toward zero
r_itt   <- rdrobust(y = yy, x = xx, c = 0)
## fuzzy: scaled up by the first stage
r_fuzzy <- rdrobust(y = yy, x = xx, c = 0, fuzzy = takeup)

c(ITT = r_itt$coef[1], fuzzy = r_fuzzy$coef[1], truth = 1.5)
#>    ITT  fuzzy  truth
#> 0.8771 1.4247 1.5000
```

The sharp estimate is the *intention-to-treat* effect of crossing the cutoff. The fuzzy estimate divides it by the jump in take-up, exactly as in Week 7 — and it inherits the LATE interpretation. Here the estimand is the effect for compliers *at the cutoff*.

8 Exercises

1. **Report an RD properly.** Write a short results paragraph for the Senate application containing: the robust estimate and interval, the bandwidth and effective n, the density test, one placebo, and one sentence stating the estimand. Under 150 words.
2. **Kernel choice.** Re-estimate with `kernel = "uniform"` and `kernel = "epanechnikov"`. How much does the answer move relative to its standard error? Which choice matters more, kernel or bandwidth?
3. **Local polynomial order.** Compare `p = 1`, `p = 2`, and `p = 3` in `rdrobust`. Note both the estimate and the selected bandwidth. Why does the optimal bandwidth widen with p?
4. **Covariates.** `rdrobust` accepts a `covs` argument. Add one and see what happens to the standard error and the point estimate. Under what condition is adding covariates to an RD legitimate?
5. **Detecting your own manipulation.** In `manip-sim`, vary the strength of the push (the `u > 0.5` threshold). At what point does the density test start to detect it? What does that tell you about the test's power?
6. **A real design.** Find an RD in the sociological literature. Identify the running variable, the cutoff, and the estimand in one sentence each. Then list which of the six checks from lecture the paper reports.

9 Session information

```
sessionInfo()
#> R version 4.4.1 (2024-06-14)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS 26.5.2
#>
#> Matrix products: default
#> BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: Asia/Shanghai
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
```

```
#>
#> other attached packages:
#> [1] rddensity_2.6  rdrobust_3.0.0 ggplot2_4.0.0  dplyr_1.1.4
#>
#> loaded via a namespace (and not attached):
#>  [1] vctrs_0.6.5        cli_3.6.5          knitr_1.48         rlang_1.3.0
#>  [5] xfun_0.47          generics_0.1.3     S7_0.2.0           jsonlite_1.8.9
#>  [9] labeling_0.4.3     lpdensity_2.5      glue_1.7.0         htmltools_0.5.8.1
#> [13] tinytex_0.53       scales_1.4.0       fansi_1.0.6        rmarkdown_2.28
#> [17] grid_4.4.1         evaluate_1.0.0     tibble_3.2.1       MASS_7.3-60.2
#> [21] fastmap_1.2.0      yaml_2.3.10        lifecycle_1.0.4    compiler_4.4.1
#> [25] RColorBrewer_1.1-3 pkgconfig_2.0.3    farver_2.1.2       digest_0.6.37
#> [29] R6_2.5.1           tidyselect_1.2.1   utf8_1.2.4         pillar_1.9.0
#> [33] magrittr_2.0.3     withr_3.0.1        tools_4.4.1        gtable_0.3.6
```